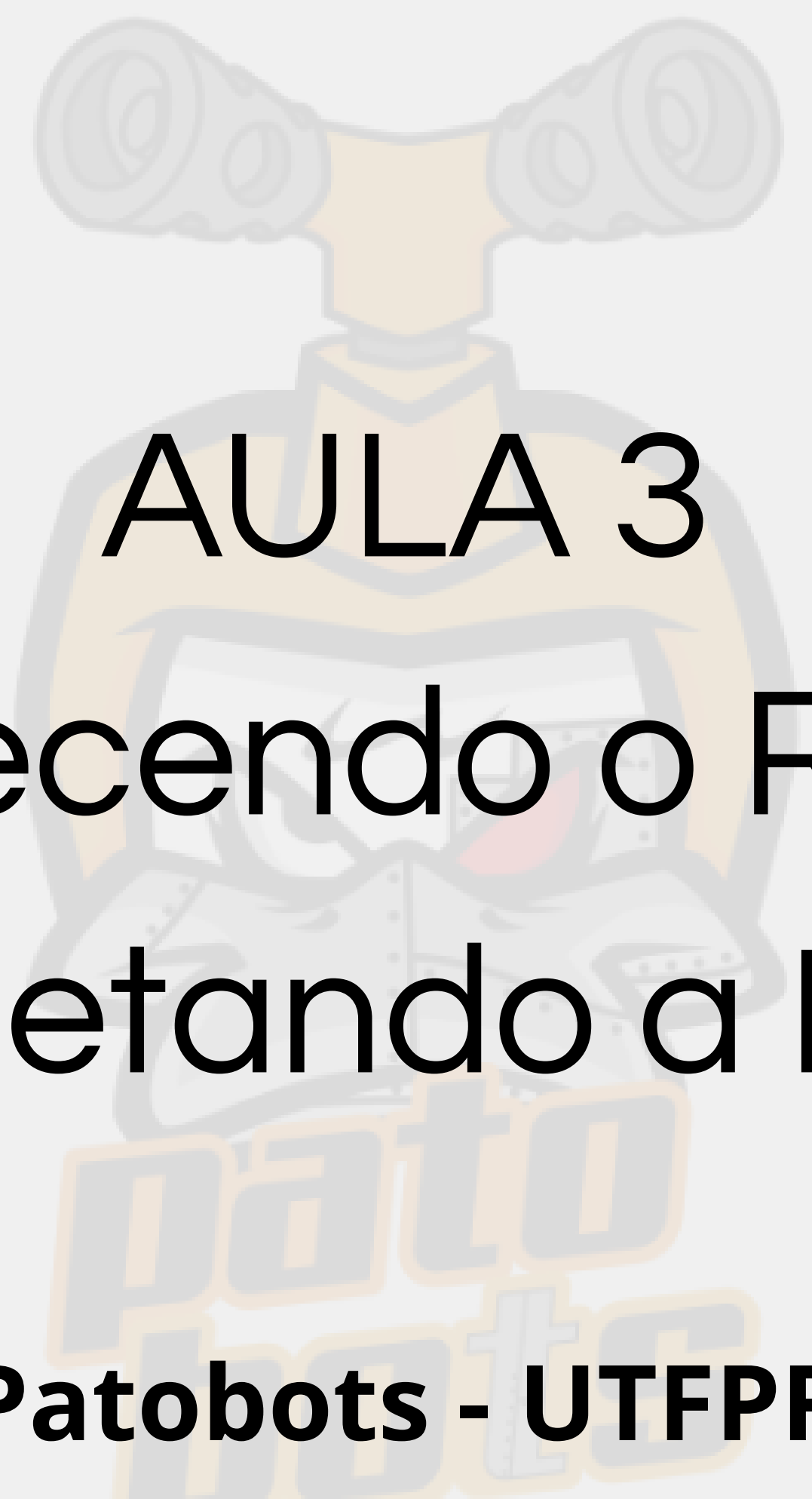




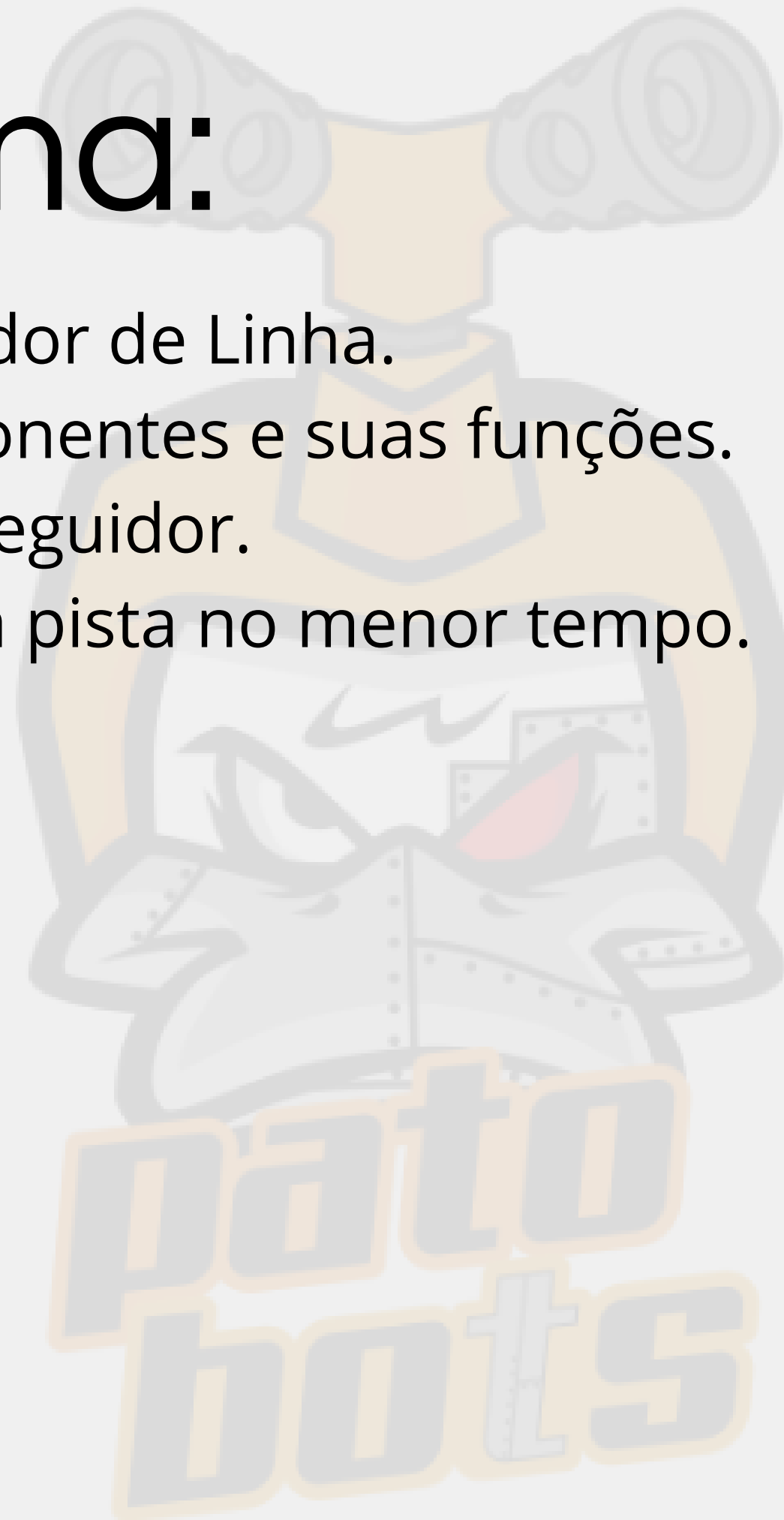
AULA 3

Conhecendo o Robô e Completando a Lógica

Patobots - UTFPR

Cronograma:

- Conhecer o Robô Seguidor de Linha.
- Compreender os componentes e suas funções.
- Completar a lógica do Seguidor.
- Completar uma volta na pista no menor tempo.

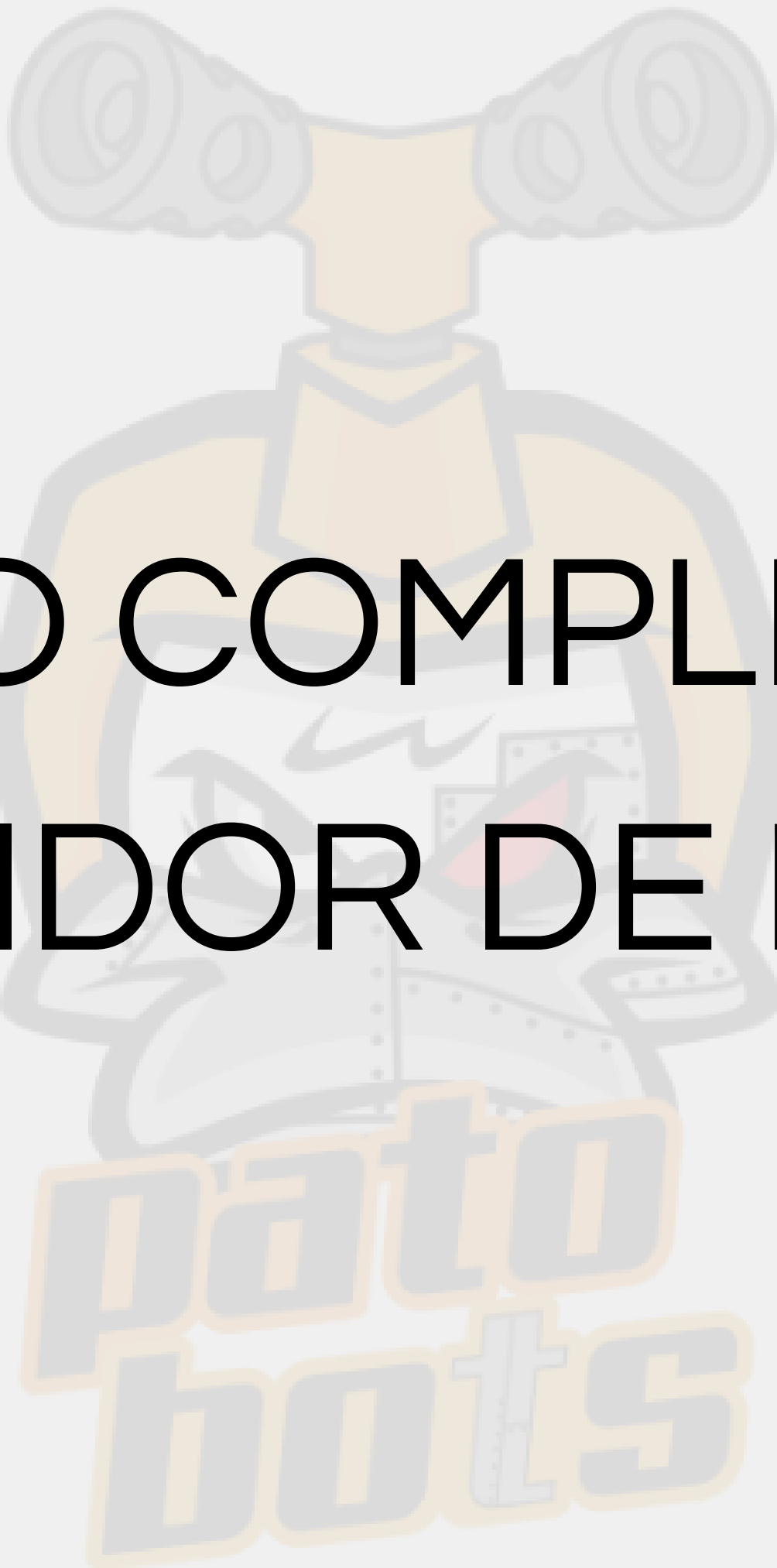


GITHUB:

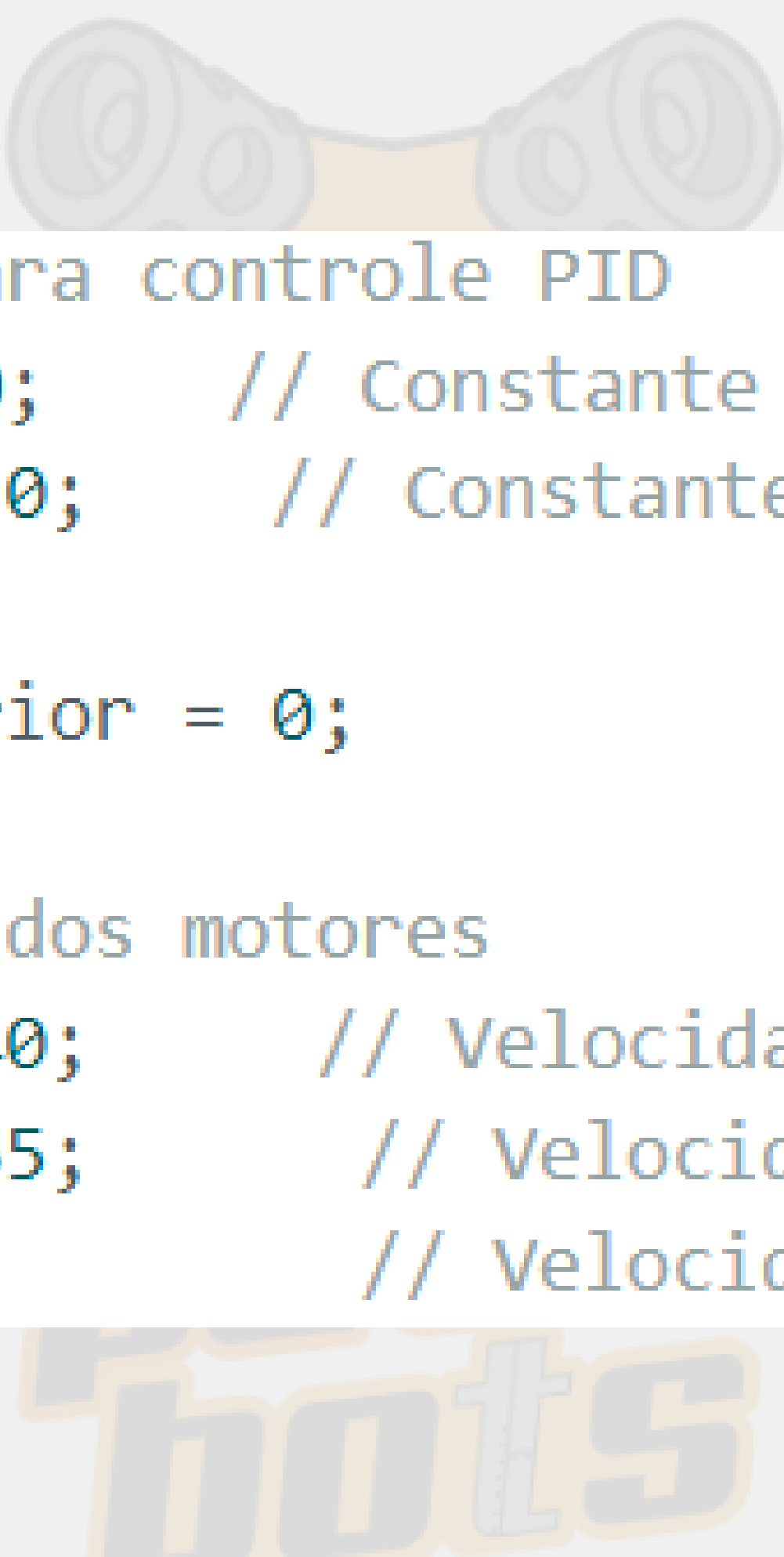


<https://github.com/PatoBots/Curso-Introducao-Robotica-Movel/tree/Curso-2025-1>

CÓDIGO COMPLETO DO SEGUIDOR DE LINHA



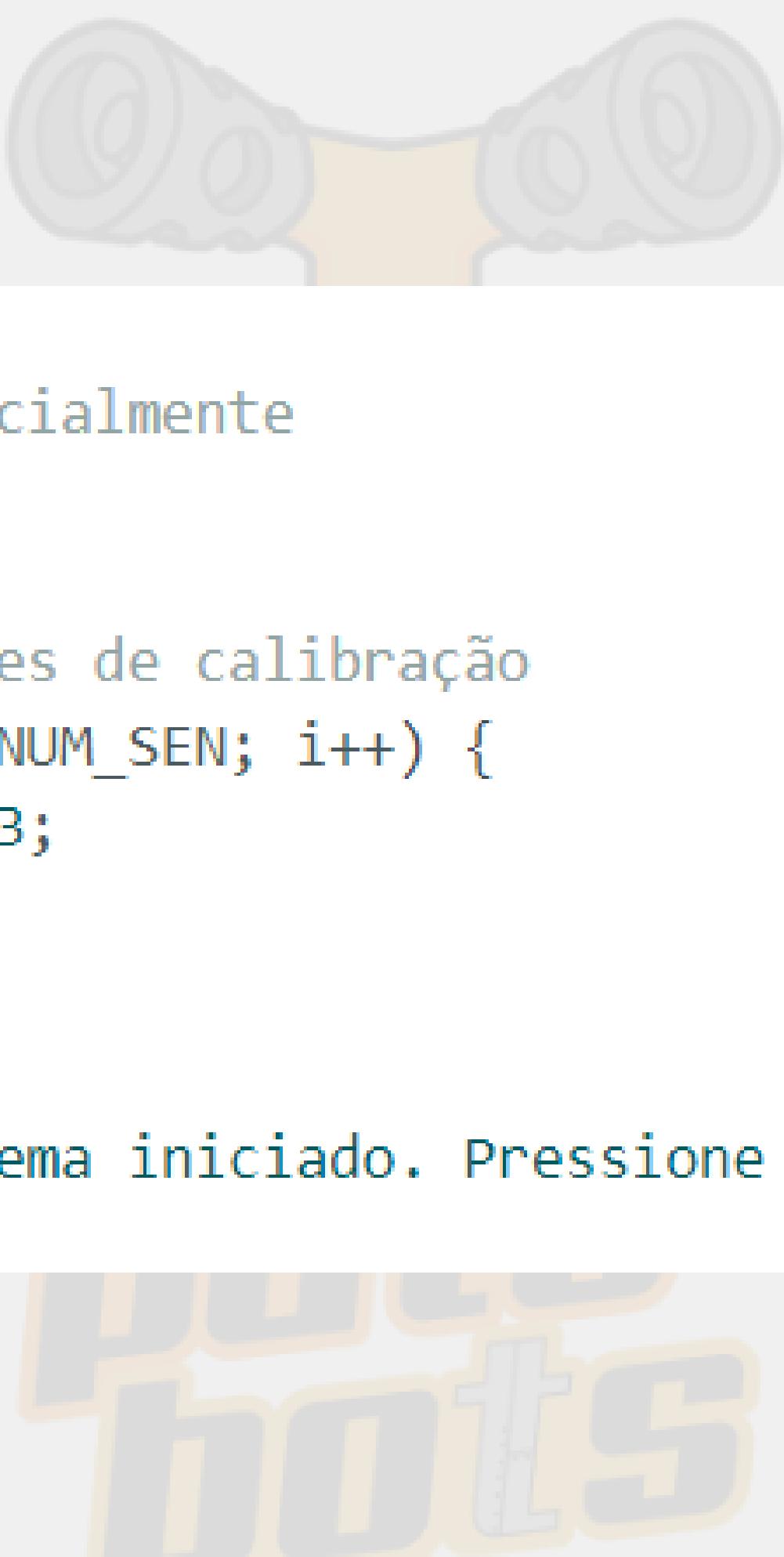
```
6 // Definições dos pinos
7 #define NUM_SEN 8
8 #define BUTTON_PIN 12
9
10 // Pinos do motor driver TB6612FNG
11 #define PWMA 5 // PWM Motor A (esquerda)
12 #define AI1 9 // Direção Motor A
13 #define AI2 4 // Direção Motor A
14 #define PWMB 6 // PWM Motor B (direita)
15 #define BI1 7 // Direção Motor B
16 #define BI2 8 // Direção Motor B
17
18 // Pinos dos sensores (A0 a A7)
19 int pinoSensores[NUM_SEN] = {A0, A1, A2, A3, A4, A5, A6, A7};
20
21 // Variáveis para calibração
22 int minSensor[NUM_SEN];
23 int maxSensor[NUM_SEN];
24 int valorSensores[NUM_SEN];
```

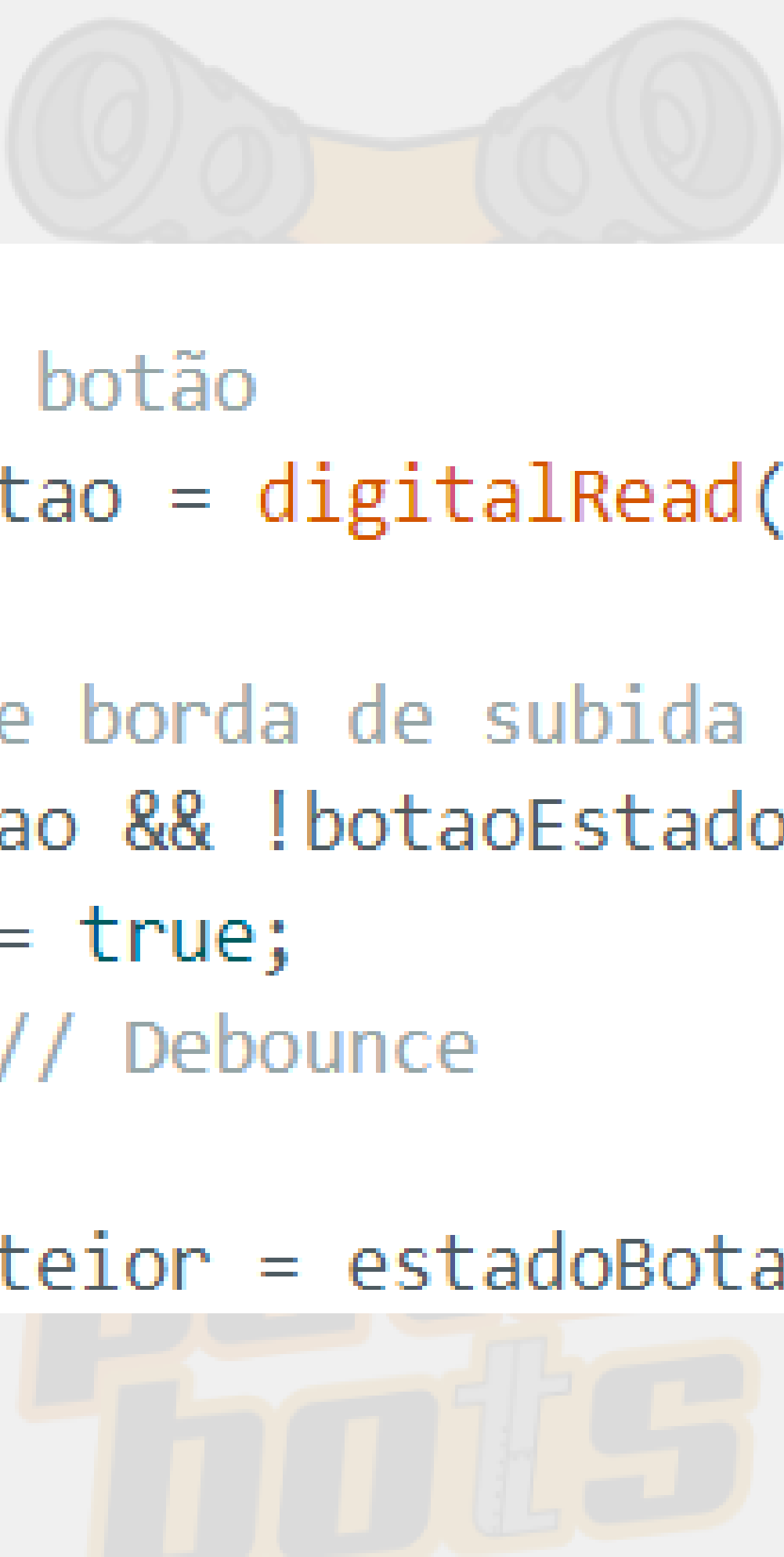
```
26 // Variáveis para controle PID
27 float Kp = 50.0;    // Constante proporcional
28 float Kd = 150.0;    // Constante diferencial
29 float erro = 0;
30 float erroAnterior = 0;
31
32 // Velocidades dos motores
33 int velBase = 40;    // Velocidade base (0-255)
34 int velMax = 255;    // Velocidade máxima
35 int velMin = 0;      // Velocidade mínima
```

```
38 // Estados do programa
39 enum Estado {
40     ESPERANDO,
41     CALIBRANDO,
42     CORRENDO
43 };
44
45 Estado estadoAtual = ESPERANDO;
46 bool botaoAtual = false;
47 bool botaoEstadoAnterior = false;
```

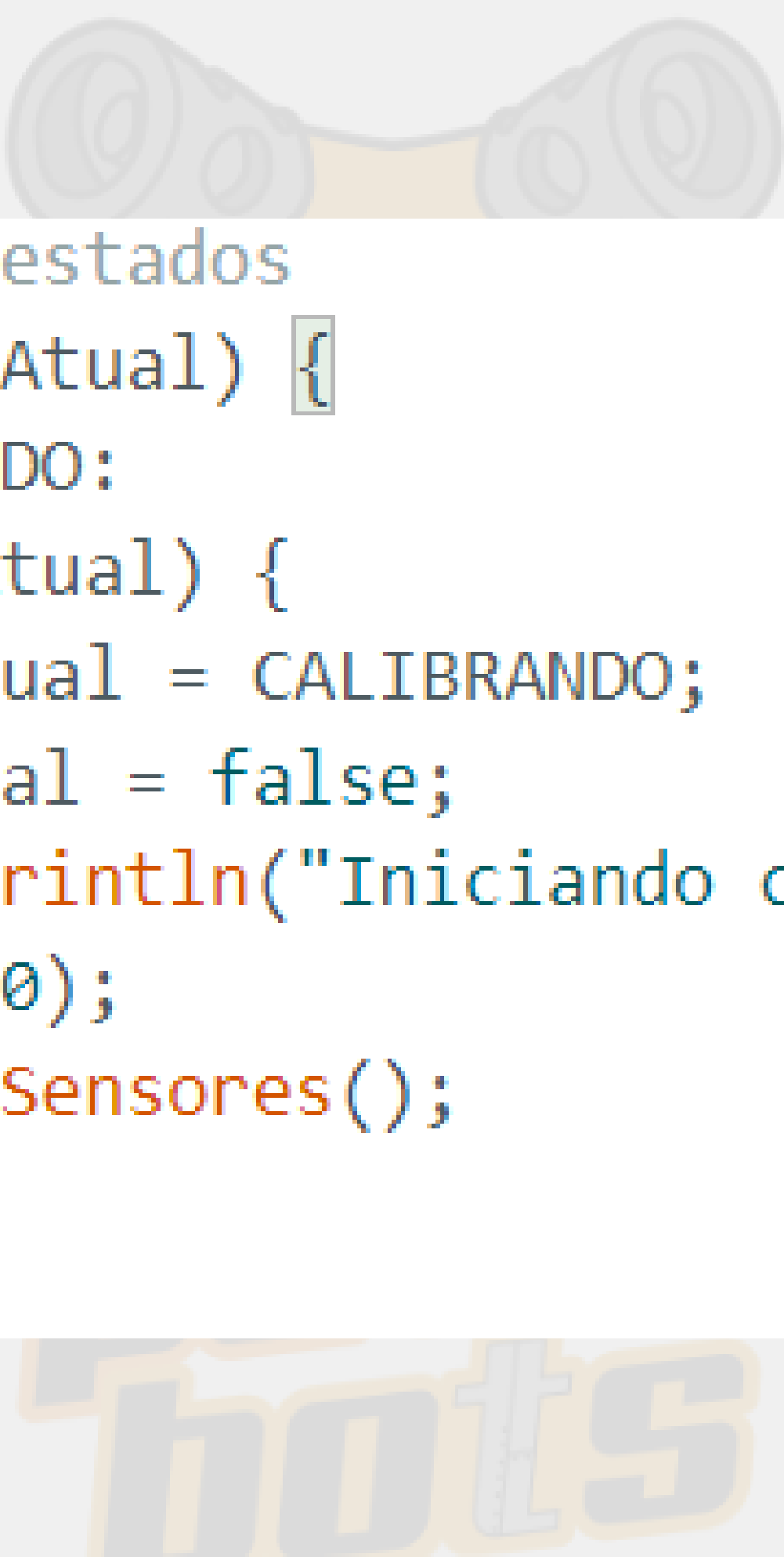
```
49 void setup() {  
50     Serial.begin(9600);  
51  
52     // Configuração dos pinos  
53     pinMode(BUTTON_PIN, INPUT);  
54  
55     // Configuração dos pinos do motor  
56     pinMode(PWMA, OUTPUT);  
57     pinMode(AI1, OUTPUT);  
58     pinMode(AI2, OUTPUT);  
59     pinMode(PWMB, OUTPUT);  
60     pinMode(BI1, OUTPUT);  
61     pinMode(BI2, OUTPUT);
```

```
62
63 // Parar motores inicialmente
64 pararMotores();
65
66 // Inicializar valores de calibração
67 for (int i = 0; i < NUM_SEN; i++) {
68     minSensor[i] = 1023;
69     maxSensor[i] = 0;
70 }
71
72 Serial.println("Sistema iniciado. Pressione o botão para calibrar.");
73 }
```



```
75 void loop() {  
76     // Leitura do botão  
77     bool estadoBotao = digitalRead(BUTTON_PIN);  
78  
79     // Detecção de borda de subida do botão  
80     if (estadoBotao && !botaoEstadoAnterior) {  
81         botaoAtual = true;  
82         delay(50); // Debounce  
83     }  
84     botaoEstadoAnterior = estadoBotao;
```



```
86 // Máquina de estados
87 switch (estadoAtual) {
88     case ESPERANDO:
89         if (botaoAtual) {
90             estadoAtual = CALIBRANDO;
91             botaoAtual = false;
92             Serial.println("Iniciando calibração...");
93             delay(500);
94             calibrarSensores();
95         }
96         break;
```

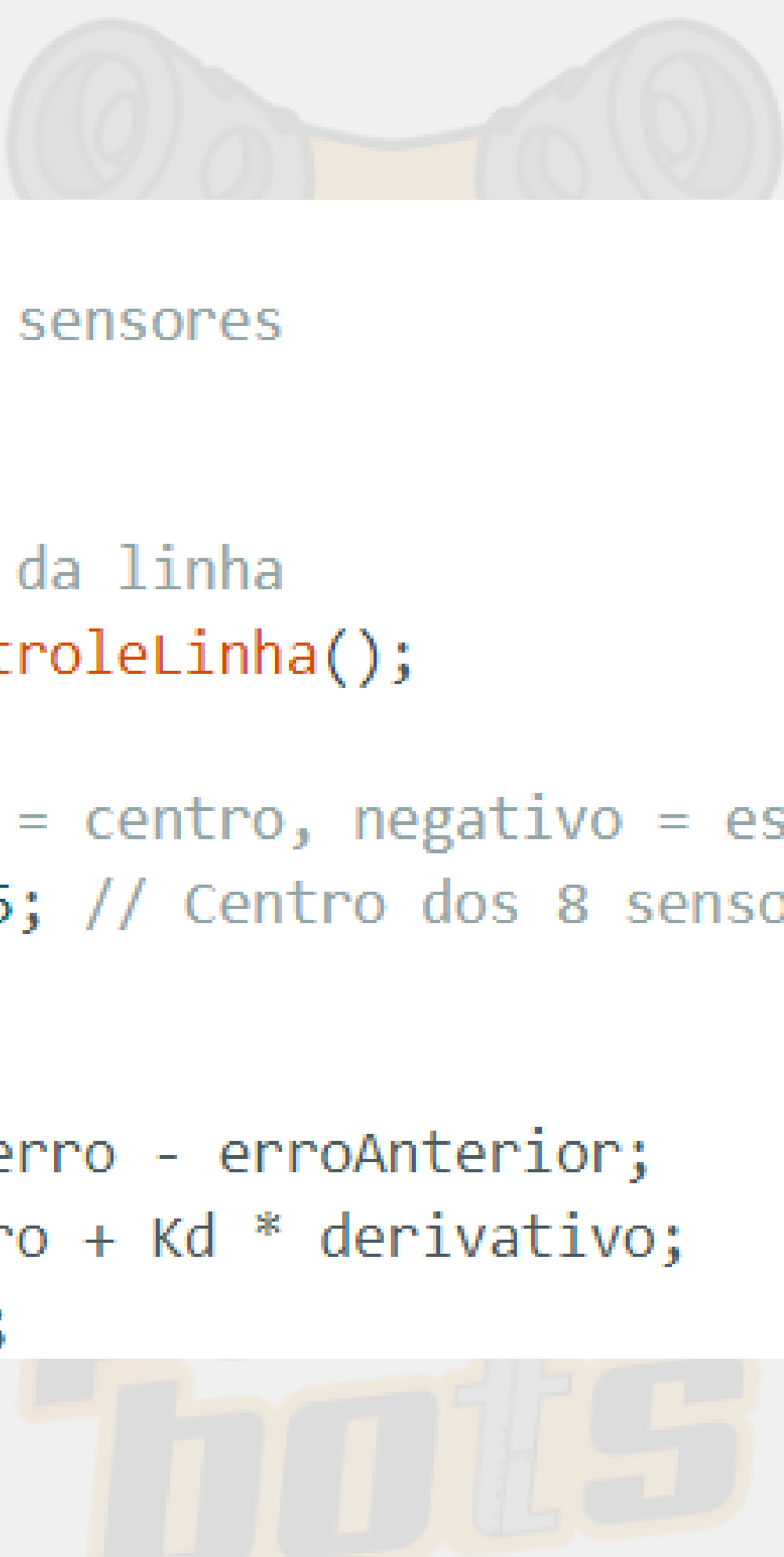
```
98     case CALIBRANDO:
99         estadoAtual = CORRENDO;
100         Serial.println("Calibração concluída. Iniciando corrida!");
101         delay(3000); // Depois de 3s inicia a corrida
102         break;
103
104     case CORRENDO:
105         if (botaoAtual) {
106             estadoAtual = ESPERANDO;
107             botaoAtual = false;
108             pararMotores();
109             Serial.println("Parando robô. Pressione o botão novamente para r
110         } else {
111             seguirLinha();
112         }
113         break;
114     }
115 }
```

```
117 void calibrarSensores() {
118     Serial.println("Movendo robô para calibração...");
119
120     // Calibração por 3 segundos girando o robô
121     unsigned long tempoInicial = millis();
122
123     while (millis() - tempoInicial < 3000) {
124         // Girar o robô lentamente para a direita
125         acelerar(50, -50);
126
127         // Ler sensores e atualizar valores min/max
128         for (int i = 0; i < NUM_SEN; i++) {
129             int faixa = analogRead(pinoSensores[i]);
130             if (faixa < minSensor[i]) {
131                 minSensor[i] = faixa;
132             }
133             if (faixa > maxSensor[i]) {
134                 maxSensor[i] = faixa;
135             }
136         }
137         delay(10);
138     }
```

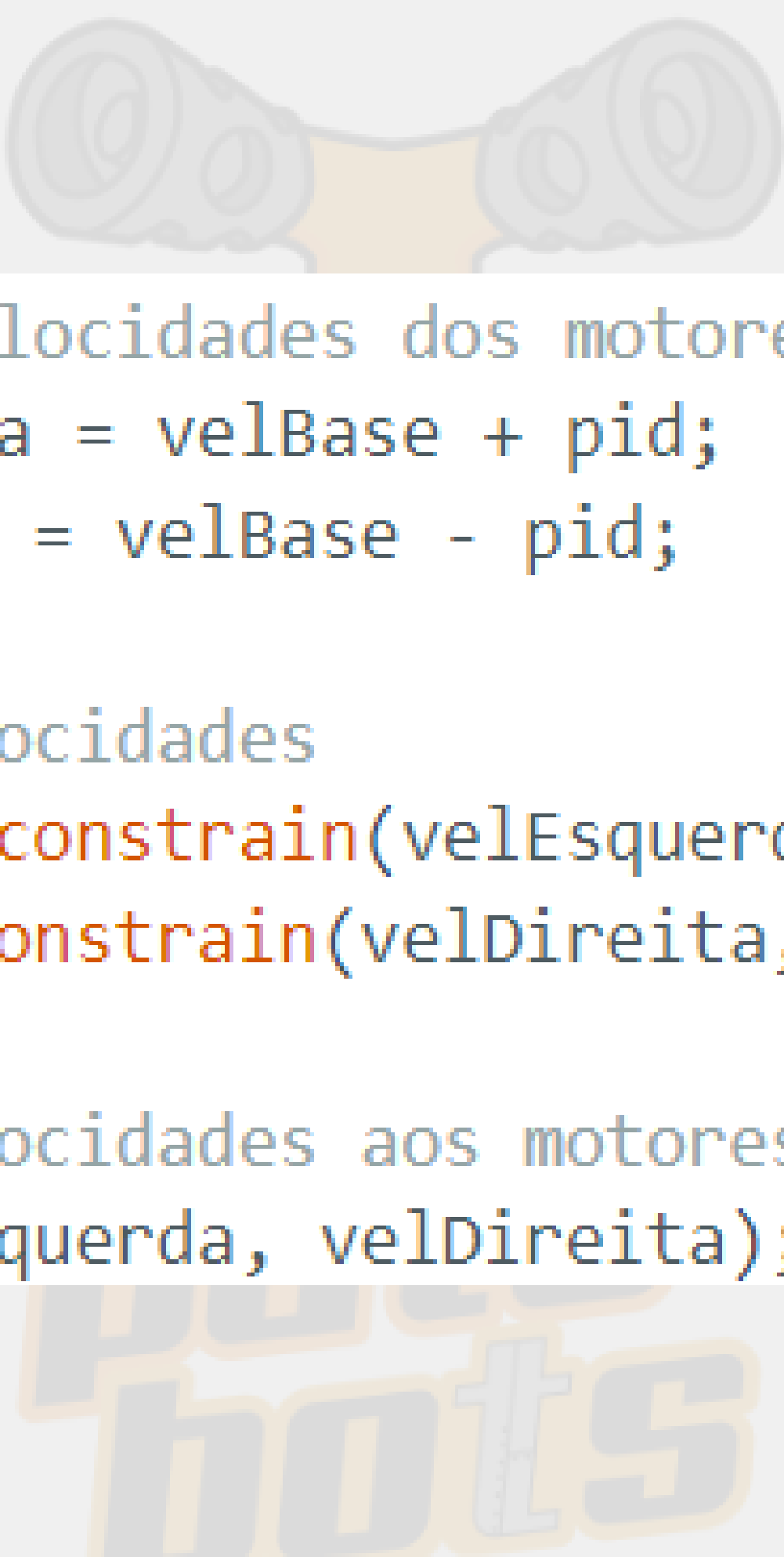


```
140 // Girar para o outro lado
141 tempoInicial = millis();
142 while (millis() - tempoInicial < 3000) {
143     // Girar o robô lentamente para a esquerda
144     acelerar(-50, 50);
145
146     // Ler sensores e atualizar valores min/max
147     for (int i = 0; i < NUM_SEN; i++) {
148         int faixa = analogRead(pinoSensores[i]);
149         if (faixa < minSensor[i]) {
150             minSensor[i] = faixa;
151         }
152         if (faixa > maxSensor[i]) {
153             maxSensor[i] = faixa;
154         }
155     }
156     delay(10);
157 }
158
159 pararMotores();
```

```
161 // Mostrar valores de calibração
162 Serial.println("Valores de calibração:");
163 for (int i = 0; i < NUM_SEN; i++) {
164     Serial.print("Sensor ");
165     Serial.print(i);
166     Serial.print(": Min=");
167     Serial.print(minSensor[i]);
168     Serial.print(", Max=");
169     Serial.println(maxSensor[i]);
170 }
171
172 delay(1000);
173 }
```



```
174 void seguirLinha() {
175     // Ler e normalizar sensores
176     lerSensores();
177
178     // Calcular posição da linha
179     float posicao = controleLinha();
180
181     // Calcular erro (0 = centro, negativo = esquerda, positivo = direita)
182     erro = posicao - 3.5; // Centro dos 8 sensores (0-7)
183
184     // Controle PID
185     float derivativo = erro - erroAnterior;
186     float pid = Kp * erro + Kd * derivativo;
187     erroAnterior = erro;
```

```
191 // Calcular velocidades dos motores
192 int velEsquerda = velBase + pid;
193 int velDireita = velBase - pid;
194
195 // Limitar velocidades
196 velEsquerda = constrain(velEsquerda, velMin, velMax);
197 velDireita = constrain(velDireita, velMin, velMax);
198
199 // Aplicar velocidades aos motores
200 acelerar(velEsquerda, velDireita);
```



```
217 void lerSensores() {  
218     for (int i = 0; i < NUM_SEN; i++) {  
219         int leitura = analogRead(pinoSensores[i]);  
220         // Normalizar entre 0 e 1000  
221         valorSensores[i] = map(leitura, minSensor[i], maxSensor[i], 0, 1000);  
222         valorSensores[i] = constrain(valorSensores[i], 0, 1000);  
223     }  
224 }
```



```
226 float controleLinha() {
227     long num = 0;
228     long den = 0;
229
230     for (int i = 0; i < NUM_SEN; i++) {
231         num += (long)valorSensores[i] * i * 1000;
232         den += valorSensores[i];
233     }
234
235     if (den == 0) {
236         return 3.5; // Retorna centro se nenhum sensor detectar linha
237     }
238
239     return (float)num / den / 1000.0;
240 }
```

```
242 void acelerar(int velEsquerda, int velDireita) {
243     // Motor esquerdo (canal A)
244     if (velEsquerda >= 0) {
245         digitalWrite(AI1, HIGH);
246         digitalWrite(AI2, LOW);
247         analogWrite(PWMA, velEsquerda);
248     } else {
249         digitalWrite(AI1, LOW);
250         digitalWrite(AI2, HIGH);
251         analogWrite(PWMA, -velEsquerda);
252     }
253
254     // Motor direito (canal B)
255     if (velDireita >= 0) {
256         digitalWrite(BI1, HIGH);
257         digitalWrite(BI2, LOW);
258         analogWrite(PWMB, velDireita);
259     } else {
260         digitalWrite(BI1, LOW);
261         digitalWrite(BI2, HIGH);
262         analogWrite(PWMB, -velDireita);
263     }
264 }
```

```
266 void pararMotores() {
267     analogWrite(PWMA, 0);
268     analogWrite(PWMB, 0);
269     digitalWrite(AI1, LOW);
270     digitalWrite(AI2, LOW);
271     digitalWrite(BI1, LOW);
272     digitalWrite(BI2, LOW);
273 }
```